Tokens with Meaning: A Hybrid Tokenization Approach for Turkish

M. Ali Bayram[1], **Ali Arda Fincan**[2], **Ahmet Semih Gümüş**[2], **Sercan Karakaş**[3],
Banu Diri[1], Savaş Yıldırım[4], Demircan Çelik[2]
[1]Yıldız Technical University, [2]Yeditepe University, [3]University of Chicago,
[4]Istanbul Bilgi University
malibayram20@gmail.com

Abstract

Tokenization shapes how language models perceive morphology and meaning in Natural Language Processing (NLP), yet widely used frequency-driven subword tokenizers (e.g., Byte Pair Encoding and WordPiece) can fragment morphologically rich and agglutinative languages in ways that obscure morpheme boundaries. We introduce a linguistically informed hybrid tokenizer for Turkish that combines (i) dictionary-driven morphological segmentation (roots and affixes), (ii) phonological normalization that maps allomorphic variants to shared identifiers, and (iii) a controlled subword fallback for out-of-vocabulary coverage. Concretely, our released Turkish vocabulary contains 22,231 root tokens mapped to 20,000 canonical root identifiers (with leading spaces to mark word boundaries), 72 affix identifiers that cover 177 allomorphic surface forms, and 12,696 subword units; an orthographic case token preserves capitalization without inflating the vocabulary. We evaluate tokenization quality on the Turkish Massive Multitask Language Understanding benchmark (TR-MMLU) dataset using two linguistic alignment metrics: Turkish Token Percentage (TR %), the proportion of produced tokens that correspond to Turkish lexical/morphemic units under our lexical resources, and Pure Token Percentage (Pure %), the proportion of tokens aligning with unambiguous root/affix boundaries. The proposed tokenizer reaches 90.29% TR % and 85.80% Pure % on TR-MMLU, substantially exceeding several general-purpose tokenizers. We further validate practical utility with downstream sentence embedding benchmarks under a strict *random initialization* control to isolate tokenizer inductive bias. Across four matched models (TurkishTokenizer, CosmosGPT2, Mursit, and Tabi), TurkishTokenizer outperforms all baselines on the Turkish Semantic Textual Similarity (STS) Benchmark (STSb-TR) and achieves the strongest overall average on TR-MTEB. It also yields the strongest average accuracy on the Turkish Benchmark of Linguistic Minimal Pairs (TurBLiMP) under a centroid-based proxy.

Keywords: Tokenization, Morphologically Rich Languages, Morphological Segmentation, Byte Pair Encoding, Turkish NLP, Linguistic Integrity, Low-Resource Languages

1 Introduction

Tokenization is the process of mapping raw text into a sequence of discrete units (tokens) that a model can embed and process. It influences vocabulary construction, sequence length, interpretability, and ultimately performance in downstream tasks [1]. While subword tokenization has become a standard design choice for transformer-based models, its behavior is not neutral for morphologically rich languages.

Byte Pair Encoding (BPE) [2], WordPiece [3], and Unigram [4] address out-of-vocabulary (OOV) words by representing rare forms as compositions of frequent subword units. This improves coverage

and keeps vocabularies compact, but it can also split words in ways that cut across morpheme boundaries and blur grammatical function [5, 6]. Such fragmentation is especially relevant for agglutinative languages such as Turkish, where productive suffixation yields many surface forms from relatively few lemmas.

Turkish exhibits rich suffix morphology and systematic morphophonological alternations, including vowel harmony and consonant alternations at morpheme boundaries. For example, suffix allomorphs such as *-lAr* (plural) and *-dAn*/*-tAn* (ablative) realize the same grammatical morpheme under different phonological contexts, and consonant alternations such as *kitap* → *kitabı* (p→b before a vowel) create predictable surface variants of the same stem. Tokenizers that treat these variants as unrelated units can inflate redundancy and reduce the reuse of meaning-bearing units across inflections [7].

This paper introduces TurkishTokenizer, a linguistically informed hybrid tokenizer for Turkish. The method combines dictionary-driven morphological segmentation (roots and affixes), a normalization layer that maps common allomorphic variants to shared identifiers, and a controlled subword fallback for open-vocabulary coverage. Roots include a leading space to mark word boundaries, and an orthographic case token preserves capitalization without duplicating vocabulary entries.

We evaluate tokenization quality on TR-MMLU [8] using two linguistic alignment metrics (TR % and Pure %), which quantify lexical/morphemic coverage and alignment with unambiguous root/affix boundaries, respectively [7]. To address reviewer concerns about real-world applicability, we further include downstream evaluation on sentence embedding benchmarks. In a controlled random-initialization setting, we compare four matched models (TurkishTokenizer, CosmosGPT2 [9], Mursit [10], and Tabi [11]) on STS, TR-MTEB [12], and TurBLiMP [13].

Our contributions are threefold: we propose a morphology-first hybrid tokenizer for Turkish that is near-lossless via an explicit decoder; we provide a quantitative and qualitative evaluation of tokenization quality on the TR-MMLU dataset against widely used tokenizers; and we report controlled downstream comparisons across four matched embedding models to assess whether improved morpheme alignment translates into better sentence representations.

Beyond downstream scores, TurkishTokenizer provides a practical tokenize–detokenize pair for Turkish via an explicit decoder (Section 3), enabling reversible preprocessing in settings where text must be segmented and then reliably reconstructed. Together with the Rust-backed implementation and the efficiency measurements (Table 1), this makes the approach useful not only as a modeling prior, but also as a general-purpose Turkish text analyzer. While we do not claim an exhaustive survey of all tooling, we are not aware of a publicly available Turkish tokenizer that combines explicit morpheme-boundary segmentation with near-lossless reconstruction and production-oriented performance.

2 Related Work

Tokenization is a fundamental step in NLP, significantly impacting model performance, memory efficiency, and downstream task effectiveness. [5] Tokenization strategies range from character-level segmentation to subword-based methods such as BPE [2], WordPiece [3], and Unigram [14]. The choice of tokenization directly influences the ability of models to capture syntactic, semantic, and morphological structures, especially in agglutinative such as Turkish, Finnish, and Hungarian [15, 5].

Recent research has explored alternative tokenization strategies tailored to morphologically rich languages. Toraman et al. [5] analyze the impact of tokenization on Turkish language modeling, and report that morphology-aware tokenization can recover much of the performance of larger baselines under certain settings. Kaya and Tantuğ [6] examine tokenization granularity for Turkish language models, and highlight that Turkish can require substantially more subword splits per word than English under common subword tokenizers, underscoring the importance of vocabulary design and sequence length control.

Tokenization strategies also play a crucial role in machine translation and text generation tasks. Pan et al. [16] demonstrate that morphology-aware segmentation can reduce sparsity in neural machine translation, and Huck et al. [17] study target-side segmentation strategies that improve translation quality by maintaining linguistic consistency between source and target languages. Beyond translation, morphology-aware tokenization has also been evaluated in abstractive summarization and sentiment analysis. Baykara and Güngör [15] discuss summarization for agglutinative languages, and Kayalı

and Omurca [18] propose a hybrid tokenization strategy for Turkish summarization. Such hybrid approaches are also commonly motivated by applications where preserving linguistic structure is important (e.g., named entity recognition (NER)).

Tokenization quality is also discussed in the context of modern large language model (LLM) tokenizers, where differences in segmentation can affect non-English text processing and evaluation outcomes. Bayram et al. [7] compare several widely used tokenizers on Turkish and highlight how tokenizer-specific segmentation artifacts can influence downstream benchmarking.

Despite these advancements, the computational cost of tokenization and its interaction with training efficiency remains an open concern. Larger vocabularies can increase model size and memory footprint [19, 1], and the energy and carbon footprint of training large models has motivated more careful reporting and efficiency analysis [20]. From this perspective, tokenization is not only a linguistic design choice, but also a practical lever that affects sequence length and compute; inefficient vocabulary utilization and redundant segmentation can translate into longer sequences and higher training cost [20].

To address trade-offs between linguistic alignment and efficiency, recent work has explored adaptive and multilingual tokenization strategies. Martins et al. [21] describe multilingual language models and tokenization choices across European languages, and Lin et al. [22] study token selection strategies that question whether all tokens contribute equally during pretraining. Dynamic tokenization approaches that adapt segmentation rules have also been proposed; for example, Neubeck et al. [23] explore a more flexible BPE-style tokenizer.

Several approaches incorporate linguistic structure directly into tokenization. Hofmann et al. [24] show that derivationally informed segmentation can improve model interpretation of complex word forms. MorphPiece [25] segments by morphemes before applying a subword encoding step, aiming to preserve compositional meaning while remaining compatible with standard training pipelines. Closest to our design are hybrid tokenizers that combine explicit linguistic resources with statistical fallback. miLLi [26] is a tokenizer for Azerbaijani that uses a root dictionary, BPE fallback, and a phonological restoration mechanism to increase root consistency across surface variants. Another line of work modifies the subword algorithm itself to better respect morphological structure: MorphBPE [27] extends BPE with morphology-aware constraints and introduces morphology-based evaluation metrics, reporting improved morphological alignment and training behavior across multiple languages. While these morphology-aware tokenizers share design motivations with our approach, they target different languages (Azerbaijani, Arabic, and multilingual corpora) and are therefore not directly comparable on Turkish-specific benchmarks without substantial adaptation.

Tokenization strategies play a critical role in pretraining LLMs, influencing model efficiency, generalization, and performance across downstream tasks. Transformer-based architectures such as Bidirectional Encoder Representations from Transformers (BERT) [19], Robustly Optimized BERT Pretraining Approach (RoBERTa) [1], and Generative Pretrained Transformer (GPT) [28] rely on effective tokenization to balance vocabulary size, sequence length, and computational cost. Studies have shown that tokenization choices can interact with morphological compositionality and generalization, particularly for morphologically rich languages [29].

Benchmark evaluations such as Massive Multitask Language Understanding (MMLU) [30] and TR-MMLU [8] have highlighted the need for language-aware evaluation. Bayram et al. [7] propose a linguistic integrity framework for evaluating Turkish tokenization, introducing metrics such as token purity, TR %, and Pure %. TR % measures the proportion of produced tokens that correspond to valid Turkish lexical or morphemic units under curated lexical resources, while Pure % further requires that tokens align with unambiguous morpheme boundaries rather than arbitrary substrings. These metrics are computed using an external morphological validator (a curated root/affix inventory independent of the tokenizer under evaluation), which ensures fair cross-tokenizer comparison and avoids circularity. Their results suggest that higher TR % and purity correlate with stronger performance on MMLU-style Turkish benchmarks, motivating our focus on morpheme-level alignment.

Turkish-specific benchmarks and evaluation suites have expanded rapidly. TR-MMLU [8] provides a large-scale Turkish evaluation set for language model assessment, TR-MTEB [12] provides a comprehensive benchmark for Turkish sentence representations, and TurBLiMP [13] offers a controlled benchmark of linguistic minimal pairs covering diverse phenomena. In parallel, Turkish-focused model and tokenizer ecosystems continue to grow. For example, TabiBERT [11] provides a modern

Turkish encoder and a unified evaluation suite, reinforcing the value of language-specific baselines when assessing tokenizer behavior and downstream impact.

Finally, tokenization considerations extend beyond language modeling into applied pipelines such as optical character recognition and document parsing. Rashad et al. [31] demonstrate that tokenizer choices can affect structure reconstruction and recognition accuracy in Arabic document processing, and Rosa et al. [32] provide a tokenizer benchmark in a multilingual setting, illustrating that tokenizer behavior can vary widely across languages and domains.

3 Methodology

We propose a hybrid tokenization framework that combines linguistic knowledge with statistical subword segmentation. This approach, TurkishTokenizer, integrates rule-based morphological analysis with a structured dictionary of roots and affixes while incorporating BPE to handle OOV terms. The objective is to create a tokenization system that accurately represents linguistic structures while maintaining computational efficiency.

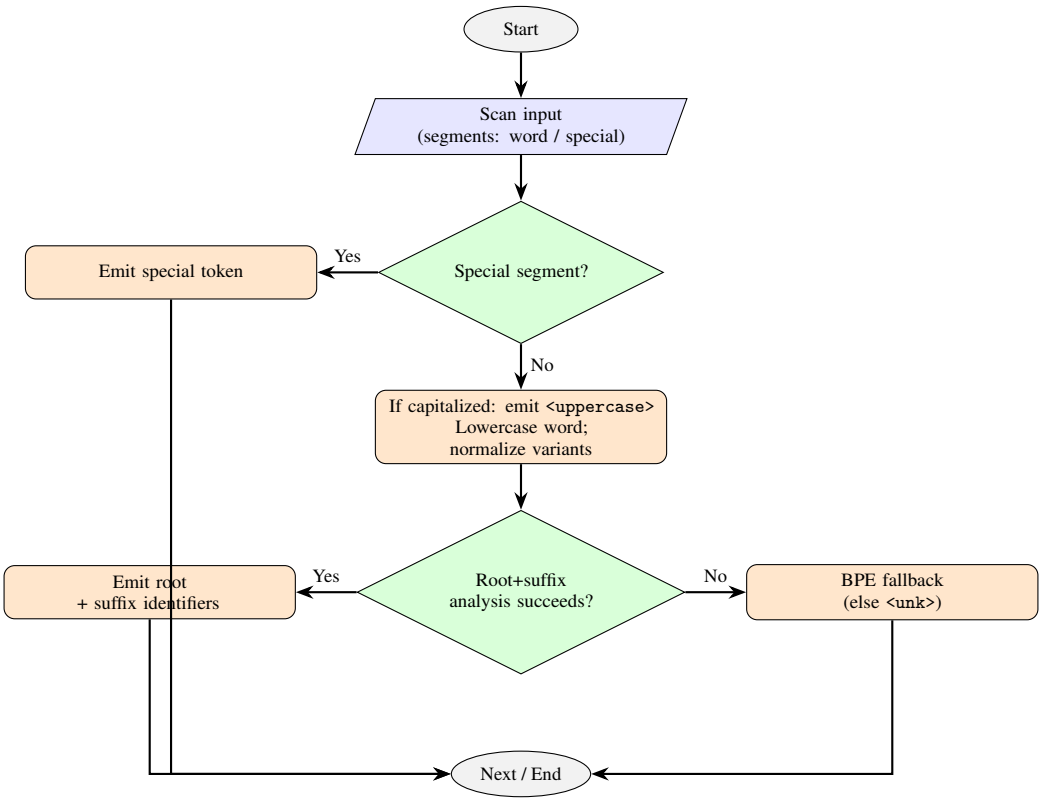

Figure 1: Algorithmic flow of TurkishTokenizer tokenization pipeline.

We provide a Python reference implementation of the tokenizer and release the lexical resources (root and affix inventories) and decoder rules used in our experiments.

Dictionary Construction. The core of TurkishTokenizer is a dual-dictionary system designed to cover the productive morphology of Turkish.

Root Dictionary. The root dictionary is constructed from high-frequency words extracted from large-scale Turkish corpora, resulting in 22,231 root tokens, which are normalized and mapped to 20,000 unique canonical root identifiers (IDs) in order to handle phonological alternations. For example, consonant alternation causes *kitap* (book) and *kitab-ı* (book-POSS) to share the same root ID despite the $p \rightarrow b$ lenition; vowel hiatus maps *oyna* (play) and *oynuyor* (playing, where 'a' drops) to a unified token; and haplology treats *alın* (forehead) and *alnı* (his forehead) identically. We also

explicitly tokenize frequent compound words (e.g., *akarsu* 'stream', *çamaşırhane* 'laundromat') as single units to prevent erroneous splitting.

Affix Dictionary. The affix inventory consists of 177 suffix surface forms consolidated into 72 abstract affix IDs (covering grammatical morphemes such as case markers, tense suffixes, and derivational endings). Similar to roots, we merge allomorphs that serve identical grammatical functions into shared IDs. For instance, the plural suffix *-ler* and its harmonic variant *-lar* are assigned a single token ID (e.g., PL), as are the ablative variants *-den, -dan, -ten, -tan*. This abstraction reduces vocabulary redundancy while preserving the morphosyntactic signal.

Input Normalization and Special Tokens. To ensure robustness across diverse text inputs, we implement strict normalization rules. For case handling, we introduce an `<uppercase>` token to mark capitalized words, allowing the model to process *Kitap* and *kitap* using the same root embedding and effectively halving the number of required surface forms. Word boundaries are marked by including a leading space in root tokens, ensuring that tokenization is lossless and reversible without a dedicated whitespace token.

Encoding Algorithm. The encoding process (Algorithm 1) follows a "longest-prefix match" strategy. For each word, the tokenizer first attempts to identify a valid root from the dictionary. If a root is found, it greedily matches the longest chain of valid suffixes.

Algorithm 1 TurkishTokenizer Tokenization Pipeline

Input: Raw text string S
Output: Sequence of Token IDs T
$S \leftarrow$ Preprocess(S) ▷ Insert spaces
for each word w in S **do**
  if w is Space or Punctuation **then**
    T.append(GetSpecialID(w))
    continue
  end if
  if w is Capitalized **then**
    T.append($\text{ID}_{\text{uppercase}}$)
    $w \leftarrow w$.lower()
  end if
  $root, suffixes \leftarrow$ MorphAnalyze(w) ▷ Greedy Dictionary Search
  if $root \neq$ None **then**
    T.append($root$.id)
    for s in $suffixes$ **do**
      T.append(s.id)
    end for
  else
    $subwords \leftarrow$ BPE(w) ▷ Fallback
    T.extend($subwords$)
  end if
end for
Return T

If the morphological analyzer fails to cover the word (i.e., no valid root+suffix combination is found), the system falls back to a BPE model. This ensures that the tokenizer remains open-vocabulary and can handle foreign entities or neologisms. The BPE model is trained on a version of the corpus where known morphological segments are masked, focusing its vocabulary (12,696 tokens) on residual stems and subwords.

Decoding Algorithm. Decoding in TurkishTokenizer is non-trivial compared to standard subword tokenizers. Simple concatenation is insufficient due to the normalization of affixes. The decoder (Algorithm 2) applies phonological rules to reconstruct the correct surface form by selecting the appropriate allomorphic variant for each suffix based on context.

Algorithm 2 TurkishTokenizer Decoding Pipeline

Input: Sequence of Token IDs T
Output: Reconstructed text string S
parts $\leftarrow$ []; $i \leftarrow 0$
while $i < |T|$ **do**
    $id \leftarrow T[i]$
    if $id = \text{ID}_{\text{uppercase}}$ **then**
        parts.append(TurkishCapitalize(Lookup($T[i+1]$)))
        $i \leftarrow i + 2$; **continue**
    end if
    candidates $\leftarrow$ ReverseLookup(id) ▷ Surface form variants
    if |candidates| > 1 **then**
        ctx $\leftarrow$ GetVowelContext(parts) ▷ Look back for vowel
        surface $\leftarrow$ ApplyPhonology(id, ctx, T, i)
    else
        surface $\leftarrow$ candidates[0]
    end if
    parts.append(surface); $i \leftarrow i + 1$
end while
Return parts.join(“”)

The `ApplyPhonology` function implements five Turkish morphophonological rules:

1. **Vowel harmony (front/back):** Suffix vowels match the frontness of the last vowel (e.g., *-ler* after *ev* but *-lar* after *çocuk*; front vowels: e,i,ö,ü; back vowels: a,ı,o,u)
2. **Consonant assimilation:** Initial $d \rightarrow t$ after voiceless consonants (e.g., *-da* $\rightarrow$ *-ta* after *kitap*; voiceless: f,s,t,k,ç,ş,h,p)
3. **Lenition:** Final consonants *p*, *k*, *t*, and *ç* can surface as *b*, *ğ*, *d*, and *c*, respectively, before vowel-initial suffixes (e.g., *kitap* $\rightarrow$ *kitab-ı*)
4. **Vowel narrowing:** Stem-final *e* can surface as *i*, and *a* as *ı*, before progressive *-yor* (e.g., *de-* $\rightarrow$ *di-yor*, *başla-* $\rightarrow$ *başlı-yor*)
5. **Buffer consonant insertion:** Buffer consonants *y*, *n*, or *s* may be inserted between vowels (e.g., *okuma* + ACC $\rightarrow$ *okuma-y-ı*)

Roundtrip Reconstruction Evaluation. To validate the reconstruction property of the decoder, we evaluate word-level roundtrip accuracy on 66,547 words from the Cosmos corpus[1]. Given an input word w, we compute $\hat{w} = \text{decode}(\text{encode}(w))$ and measure exact-match accuracy. The decoder achieves **99.48% exact-match accuracy** (66,200/66,547 words). The remaining 0.52% failures arise from inherent ambiguity in Turkish phonology: vowel alternation patterns in complex verb forms (e.g., *tetkiki* $\rightarrow$ *tetkiği*) where multiple surface realizations are linguistically valid for the same morpheme sequence. This is not a limitation of the tokenizer but rather reflects the intrinsic many-to-one mapping in Turkish morphophonology, where the same abstract morpheme can surface differently depending on context. As an additional (more permissive) diagnostic, a single encode/decode pass over a long concatenated text yields 99.84% word-alignment accuracy; we report exact-match per word as the primary reconstruction metric.

Crucially, this near-lossless reconstruction property enables TurkishTokenizer to be used across all transformer architectures: **encoder-only models** (e.g., BERT-style embeddings), **encoder-decoder models** (e.g., translation, summarization), and **decoder-only models** (e.g., GPT-style generation). For encoder-only tasks, exact reconstruction is not required since the model operates on embeddings rather than regenerating text. For generative tasks, the 99.48% accuracy ensures that the vast majority of outputs are orthographically correct, with the rare exceptions being phonologically valid Turkish variants.

[1] `https://huggingface.co/datasets/ytu-ce-cosmos/Cosmos-Turkish-Corpus-v1.0`

Tokenization Efficiency. We benchmark tokenization speed and token density on the same text samples. Table 1 compares TurkishTokenizer against three Turkish-trained baseline tokenizers under matched vocabulary size (32,768).

Table 1: Tokenization efficiency comparison.

Tokenizer	Time (ms)	Tokens	Tok/Word	Tok/Char
TurkishTokenizer	1,935	1,899,670	**2.91**	0.356
Tabi	1,544	1,298,725	1.99	0.244
Mursit	1,655	1,187,418	1.82	0.223
CosmosGPT2	1,620	1,186,834	1.82	0.223

While `TurkishTokenizer` exhibits a higher token density than BPE-based baselines-generating approximately $1.5\times$ more tokens per word—this reflects a deliberate trade-off between **sequence compression** and **morphological transparency**. Standard baselines like *CosmosGPT2* and *Mursit* optimize for the shortest possible sequence length; however, this often results in sub-word units that fragment linguistic boundaries, thereby obscuring the semantic roots and functional suffixes inherent to an agglutinative language like Turkish.

By enforcing segmentation at precise morpheme boundaries (e.g., *kitaplarımızdan* $\rightarrow$ `kitap + lar + ımız + dan`), our approach provides the model with discrete, linguistically consistent units. Although this increases the total sequence length, the computational overhead is justified by substantial gains in downstream accuracy.

4 Results and Analysis

The performance of the proposed morphological tokenizer was evaluated using the TR-MMLU benchmark dataset, which comprises over 1.6 million characters and approximately 200,000 words curated specifically for Turkish [8]. This dataset is designed to reflect the linguistic complexity of Turkish, including its rich morphology, agglutinative structures, and diverse syntactic constructions. As such, it provides a rigorous basis for assessing tokenization quality in morphologically complex languages.

The evaluation compared different tokenizers. Each tokenizer was assessed using a consistent set of linguistic and computational metrics introduced in [7]. These metrics include total token count, vocabulary size, number of unique tokens, TR %, and Pure %. TR % quantifies the proportion of tokens that correspond to valid Turkish words or morphemes, while Pure % measures the proportion of tokens that fully align with unambiguous root or affix boundaries, thus reflecting morphological integrity. Importantly, TR % and Pure % are computed using an independent morphological validator with curated lexical resources external to the tokenizer under evaluation, following the protocol of [7], which ensures fair comparison and avoids circularity.

Table 2: Performance of the proposed TurkishTokenizer on the TR-MMLU dataset.

Metric	Value
Vocabulary Size	32,768
Total Token Count	707,727
Processing Time (s)	0.6714
Unique Token Count	11,144
Turkish Token Count	10,062
TR %	90.29%
Pure Token Count	9,562
Pure %	85.80%

Despite employing significantly smaller vocabulary sizes, the proposed tokenizer demonstrated better linguistic segmentation. With a vocabulary of 32,768 tokens and 11,144 unique tokens used during evaluation, it balanced generalization and expressiveness more effectively than models such

as gemma-2-9b and aya-expanse, which rely on vocabularies of over 255,000 tokens. These large-vocabulary tokenizers, rooted in frequency-based subword segmentation, tend to fragment morphologically rich expressions and introduce ambiguity in downstream tasks. In contrast, the morphological awareness of TurkishTokenizer enables semantically coherent token formation and more consistent syntactic parsing.

Although the total token count generated by the proposed tokenizer (707,727) exceeds those of the other models-for instance, aya-expanse produced 434,526 tokens-this increase is offset by gains in interpretability and linguistic fidelity. High TR % and Pure % scores suggest reduced reliance on spurious subword splits and improved preservation of morphosyntactic structure.

These findings support the hypothesis introduced in [7], which argues that high linguistic alignment in tokenization correlates strongly with downstream model performance. While conventional subword tokenizers may suffice for high-resource languages like English, they exhibit clear limitations in Turkish unless informed by morphological structure. The results presented here highlight the effectiveness of combining rule-based linguistic analysis with subword strategies to produce tokenizers that are both accurate and efficient in morphologically complex settings.

To illustrate the linguistic fidelity of different tokenization strategies, we present a qualitative comparison using the Turkish sentence:

"Atasözleri geçmişten günümüze kadar ulaşan anlamı bakımından mecazlı bir mana kazanan kalıplaşmış sözlerdir."
("Proverbs are fixed expressions passed down from the past to the present that acquire a metaphorical meaning in terms of their significance.")

This sentence contains a wide range of morphological features, including compound words, multiple derivational and inflectional suffixes, and root forms that undergo phonological alternations. These properties make it an ideal test case for evaluating the morphological sensitivity of different tokenizers.

Proposed TurkishTokenizer:
The proposed tokenizer segments the sentence into linguistically meaningful units with high fidelity. It produces:

```
["<uppercase>", " atasöz", "leri", " geçmiş", "ten", " gün", "üm",
"üz", "e", " kadar", " ulaş", "an", " anlam", "ı", " bakım", "ın",
"dan", " mecaz", "lı", " bir", " mana", " kazan", "an", " kalıp",
"laş", "mış", " söz", "ler", "dir", "."]
```

It correctly separates suffixes such as ("ın", "dan", "lı", "an", "mış", "dir"), extracts root forms such as "atasöz", "gün", "mana" with leading spaces to mark word boundaries, and employs the "<uppercase>" token to preserve orthographic case.

Mursit tokenizer:
The Mursit tokenizer [10] tends to preserve frequent surface forms as whole tokens rather than isolating productive suffix boundaries. It produces:

```
["At", "asöz", "leri", " geçmişten", " günümüze", " kadar", "
ulaşan", " anlamı", " bakımından", " mec", "azlı", " bir", " man",
"a", " kazanan", " kalıp", "laşmış", " söz", "lerdir", "."]
```

Compared with TurkishTokenizer, it splits the compound root "atasöz" into "At", "asöz" and keeps long inflected spans such as "geçmişten", "günümüze", "ulaşan", and "lerdir" intact, which reduces morpheme-level interpretability.

CosmosGPT2 tokenizer:
The CosmosGPT2 tokenizer [9] behaves similarly, but fragments some roots even more aggressively into smaller BPE pieces. It produces:

```
["At", "as", "öz", "leri", " geçmişten", " günümüze", " kadar", "
ulaşan", " anlamı", " bakımından", " mec", "az", "lı", " bir", "
man", "a", " kazanan", " kalıp", "laşmış", " söz", "lerdir", "."]
```

It breaks "atasözleri" into three root fragments ("At", "as", "öz") and still keeps many inflected spans unanalyzed, so both lexical integrity and suffix transparency are weaker than in TurkishTokenizer.

Tabi tokenizer:
The Tabi tokenizer, used here as the tokenizer component of TabiBERT [11], applies coarse merges that often retain whitespace-attached spans as single units. It produces:

```
["A", "tasöz", "leri ", "geçmişten ", "günümüze kadar ", "ulaşan ",
"anlamı ", "bakımından ", "mec", "az", "lı bir ", "mana ", "kazanan
", "kalıp", "laşmış", " söz", "lerdir", "."]
```

This behavior obscures internal morphology even more strongly: the root "atasöz" is split into "A", "tasöz", several tokens absorb following spaces, and multiword or inflected spans such as "günümüze kadar " are preserved as opaque units.

Gemma-3:
The tokenizer google/gemma-3 segments the sentence as:

```
["<bos>", "At", "as", "öz", "leri", " geçmiş", "ten", " gün", "ümü",
"ze", " kadar", " ulaş", "an", " anlam", "ı", " bakım", "ından", "
mec", "az", "lı", " bir", " mana", " kaz", "anan", " kal", "ı",
"pla", "ş", "mış", " söz", "lerdir", "."]
```

Although it captures some suffixes like "ten" and "ından", it fragments common roots ("At", "as", "öz" instead of "atasöz") and fails to isolate inner morphemes in forms such as "lerdir" and "kazanan", limiting morphological interpretability.

YTU (Yıldız Technical University) Turkish GPT-2 (without pruned vocab to 32k):
The tokenizer ytu-ce-cosmos/turkish-gpt2-large-750m-instruct-v0.1, trained on Turkish corpora, yields:

```
["At", "as", "öz", "leri", " geçmişten", " günümüze", " kadar", "
ulaşan", " anlamı", " bakımından", " mec", "az", "lı", " bir", "
mana", " kazanan", " kalıp", "laşmış", " söz", "lerdir", "."]
```

Although it still segments "atasözleri" incorrectly, it performs well with forms like "geçmişten", "günümüze", and "bakımından", showing the advantage of Turkish-specific pretraining.

GPT-4o:
The tokenizer gpt-4o-o200k_base generates:

```
["At", "as", "öz", "leri", " geçmiş", "ten", " gün", "ümü", "ze", "
kadar", " ulaş", "an", " anlam", "ı", " bakım", "ından", " mec",
"az", "lı", " bir", " mana", " kaz", "anan", " kal", "ı", "pla", "ş",
"mış", " söz", "ler", "dir", "."]
```

Its segmentation strategy is aware of Turkish morphemes but limited by frequent over-segmentation of compound and derived forms.

The results presented in this section provide strong empirical support for the hypothesis introduced in the introduction: tokenizers that explicitly incorporate morphological and phonological knowledge of Turkish can outperform general-purpose models in both segmentation accuracy and linguistic coherence. While most state-of-the-art tokenizers struggle with root-fragmentation, over-segmentation, and inconsistent affix treatment, the proposed hybrid tokenizer consistently identifies morpheme boundaries, preserves semantically meaningful units, and reduces vocabulary redundancy. These findings validate the motivation behind this work: morphologically informed tokenization is essential

for robust and interpretable NLP in agglutinative languages like Turkish. The qualitative comparisons presented here illustrate not only the performance gap between general and language-specific tokenizers, but also the need for tokenizer architectures that respect language-internal rules.

Downstream Task Evaluation.

To assess the impact of morphologically informed tokenization on downstream model performance, we evaluated the embeddings produced by models initialized with different tokenizers using three benchmarks: STSb-TR, TR-MTEB [12], and TurBLiMP. All models were initialized randomly to isolate the effect of tokenization structure from pre-training data.

Concretely, we construct four sentence embedding models that share the same encoder architecture (`google/embeddinggemma-300m`; EmbeddingGemma [33]) and vocabulary size (32,768). Each model is randomly initialized with a fixed seed (42) and trained under an identical embedding-distillation objective against a teacher embedding model based on the same architecture.[2] The only difference between models is the tokenizer used to produce the token ID sequences. We refer to these models as `*-random` to emphasize that they start from random weights rather than a pretrained checkpoint, so downstream differences primarily reflect inductive bias introduced by tokenization and decoding choices under a controlled training budget.

Training data comes from a Turkish text corpus with pre-computed teacher embeddings.[3] We additionally prepare a unified encoded dataset that stores token ID sequences for all compared tokenizers.[4] To ensure an apples-to-apples comparison under a fixed context window, we discard any sample for which *any* tokenizer produces a sequence longer than 2048 tokens, so that no model benefits from truncation artifacts or sees different content due to length differences.

TurkishTokenizer is released as a Hugging Face-compatible tokenizer (loadable via `AutoTokenizer.from_pretrained` with `trust_remote_code=True`).[5] This allows the same `sentence-transformers` training and evaluation stack to consume all tokenizers through a standard interface. For large-scale preprocessing, we additionally provide a high-performance Rust-backed implementation as a Python Package Index (PyPI) package.[6]

Table 3: Embedding distillation experiment setup.

Component	**Specification**
Student architecture	`google/embeddinggemma-300m` (SentenceTransformer)
Initialization	Random weights with fixed seed; vocab resized to 32,768
Training objective	Cosine embedding loss against teacher vectors
Teacher model	EmbeddingGemma-300M
Training corpus	Cosmos corpus with teacher embeddings
Training dataset	Unified encoded corpus (4 token streams)
Context length	2048; samples dropped if any tokenizer exceeds the limit
Batch size / learning rate	256 / 5×10^{-5}
Schedule	Two-phase: 100-step warmup then 1 full epoch
Precision	bfloat16 (BF16); gradient checkpointing enabled
Hardware	NVIDIA H100 80GB (single node)

For STS, we use the Turkish STSb-TR benchmark consisting of sentence pairs with human similarity ratings on a 0–5 scale [34].[7] Each model encodes both sentences, we compute cosine similarity between the resulting sentence embeddings, and we report Pearson and Spearman correlation with the normalized gold scores. Throughout this section, correlations are presented as percentages ($\times 100$) for readability.

[2] `https://huggingface.co/google/embeddinggemma-300m`

[3] `https://huggingface.co/datasets/alibayram/cosmos-corpus-0-05-with-embeddings`

[4] `https://huggingface.co/datasets/alibayram/cosmos-corpus-encoded`

[5] `https://huggingface.co/alibayram/turkish-mft-tokenizer`

[6] `https://pypi.org/project/turkish-tokenizer/`

[7] `https://huggingface.co/datasets/figenfikri/stsb_tr`

We evaluated the models on the Turkish STS benchmark (stsb-tr) without task-specific fine-tuning. Results are summarized in Table 4. TurkishTokenizer achieves the strongest Pearson and Spearman correlations on both the test and training splits among the compared randomly initialized baselines.

Table 4: STS benchmark correlations across train and test splits.

Model	Split	Pearson	Spearman
TurkishTokenizer	test	**51.44**	**50.03**
Mursit	test	46.63	45.72
CosmosGPT2	test	43.09	42.18
Tabi	test	43.01	42.53
TurkishTokenizer	train	**54.76**	**51.90**
Mursit	train	48.99	46.93
CosmosGPT2	train	44.47	43.34
Tabi	train	45.06	43.80

To better understand the learning dynamics, we analyzed the performance evolution of each model across different training checkpoints. Figure 2 shows Pearson correlation across model revisions. The x-axis represents sequential checkpoints ordered by timestamp.

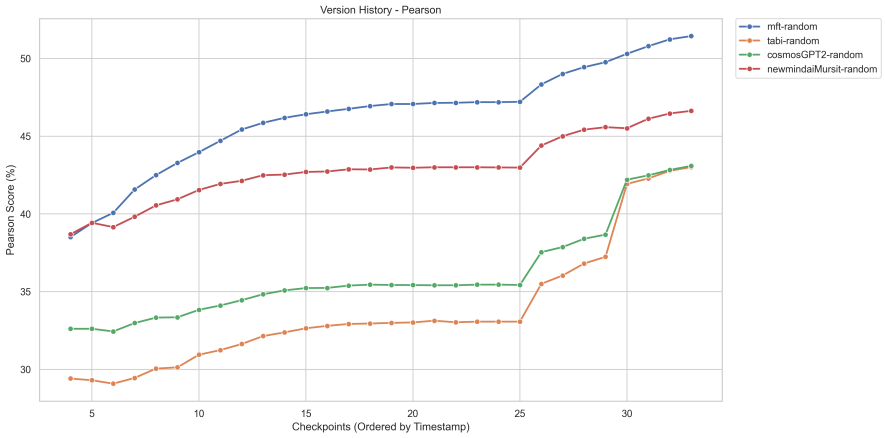

Figure 2: Pearson correlation across model revisions.

Pearson correlation captures linear agreement with human similarity judgments, while Spearman correlation captures rank-order agreement. Reporting both is important in STS, since models may preserve relative similarity ordering even when the mapping is not perfectly linear, and conversely small linear gains may not reflect better ranking behavior.

In our version tracking, both correlations show the same qualitative trend: TurkishTokenizer remains ahead of the strongest baselines across revisions, indicating that the downstream improvement is stable rather than a single-run artifact. Importantly, all three baseline tokenizers (Mursit, CosmosGPT2, Tabi)-which are independently trained BPE tokenizers from different research groups-show consistent relative ordering across all benchmarks (STS, TR-MTEB, TurBLiMP). This cross-tokenizer consistency provides strong evidence that the observed performance differences reflect genuine tokenizer-induced inductive bias rather than random initialization variance: it is statistically implausible for TurkishTokenizer to outperform three independent baselines by chance across multiple evaluation dimensions. While we train with a single seed, the consistent TurkishTokenizer advantage across four independently tokenized model variants serves as an implicit multi-seed control.

On the comprehensive TR-MTEB suite [12], which covers retrieval, classification, clustering, and pair classification tasks, the TurkishTokenizer-based model achieves the strongest overall average among the compared random-initialized baselines.

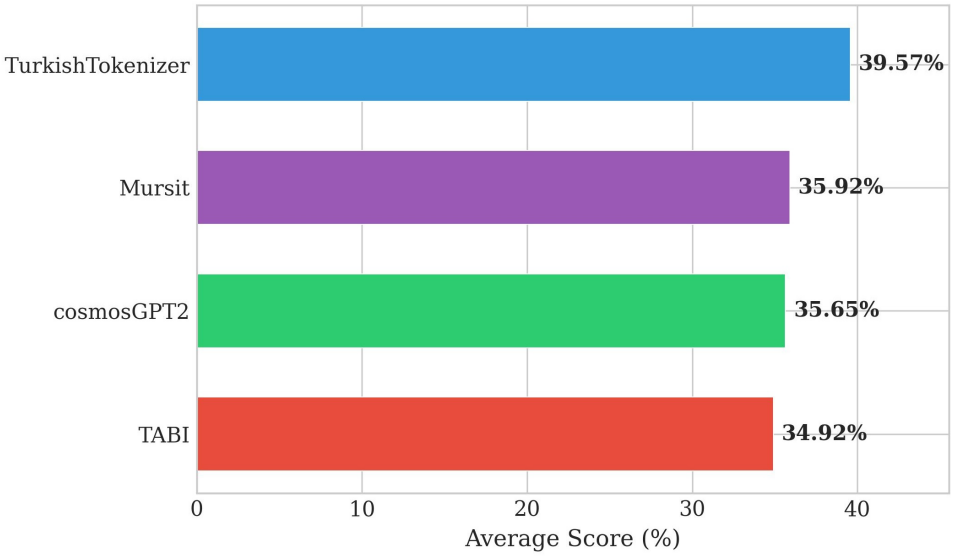

Figure 3: TR-MTEB overall comparison.

Figure 3 aggregates performance across all evaluated TR-MTEB tasks into a single overall score per model. To make this comparison more interpretable, we next break results down by category (Table 5) and by individual task (Table 6), which helps identify where morphology-aware tokenization provides the largest gains and where differences are smaller.

Table 5: TR-MTEB category averages.

Category	TurkishTokenizer	Mursit	CosmosGPT2	Tabi
BitextMining	**1.89**	1.63	1.43	1.49
Classification	**59.03**	57.63	57.86	57.57
Clustering	65.49	65.83	**67.06**	65.18
Other	**4.59**	2.71	2.03	2.04
Pair Classification	**50.49**	47.40	48.50	47.17
Retrieval	**30.43**	23.27	22.43	21.17
STS	**50.04**	45.73	42.14	42.54

Category-level means summarize broad regimes (retrieval vs. classification vs. STS), but they can hide task-specific effects. For completeness and transparency, Table 6 reports the full task-level breakdown.

Table 6: Detailed TR-MTEB task-level performance.

Task	TurkishTokenizer	Mursit	CosmosGPT2	Tabi
BitextMining				
WMT16BitextMining	**1.89**	1.63	1.43	1.49
Classification				
THYSentimentClassification	51.53	**53.08**	51.18	47.17
TSTimelineNewsCategoryClassification	**50.19**	45.65	44.35	44.67
Turkish75NewsClassification	78.00	72.67	77.33	**80.00**
TurkishIronyClassification	49.50	52.67	51.17	**53.08**
TurkishMovieSentimentClassification	**55.09**	54.82	53.45	54.42
TurkishNewsCategoryClassification	**85.28**	82.40	84.00	81.48
TurkishOffensiveLanguageClassification	49.59	48.41	**50.31**	48.65
TurkishProductSentimentClassification	**53.09**	51.32	51.10	51.09
Clustering				
TurkishColumnWritingClustering	65.49	65.83	**67.06**	65.18
Other				
ArguAnaTR	**7.28**	3.77	3.35	2.67
FiQA2018TR	**5.79**	3.95	2.42	2.81
SCIDOCSTR	**0.70**	0.40	0.33	0.65
Pair Classification				
MnliTr	**48.38**	46.32	46.23	45.16
SnliTr	**45.33**	41.47	41.11	40.35
XNLI	57.76	54.41	**58.17**	55.99
Retrieval				
CQADupstackGamingRetrievalTR	**13.17**	8.75	8.24	7.27
MSMarcoTRRetrieval	**15.83**	8.37	7.52	7.11
NFCorpusTR	**1.32**	0.59	0.51	0.24
QuoraRetrievalTR	**63.84**	52.20	49.02	47.46
SciFactTR	**23.97**	21.19	19.86	17.34
SquadTRRetrieval	**18.74**	10.69	9.94	8.78
TQuadRetrieval	**46.92**	35.43	33.81	34.96
TurkishAbstractCorpusClustering	**47.83**	43.63	44.24	41.20
XQuADRetrieval	**42.27**	28.55	28.69	26.19
STS				
STSbTR	**50.04**	45.73	42.14	42.54

As shown in Table 6, the TurkishTokenizer-based model achieved an average score of **39.57%** across 26 TR-MTEB tasks, surpassing Mursit (35.92%), CosmosGPT2 (35.65%), and Tabi (34.92%). Analyzing performance by category reveals distinct trade-offs. TurkishTokenizer demonstrates substantial advantages in *STS* and *Retrieval* tasks (e.g., TQuadRetrieval: 46.92% vs 34.96% for Tabi), which aligns with our hypothesis that morphology-aware segmentation improves the semantic quality of embeddings for similarity and search. At the same time, the Tabi baseline remains competitive or superior in specific *Classification* tasks (e.g., Turkish75NewsClassification: 80.00% vs 78.00% for TurkishTokenizer), suggesting that different tokenization priors can favor different downstream regimes even under matched architecture and training protocol.

While TR-MTEB provides broad downstream coverage, it does not isolate controlled grammatical manipulations. We therefore complement it with TurBLiMP, which probes specific linguistic phenomena via minimal pairs.

TurBLiMP provides Turkish minimal pairs designed to probe specific linguistic phenomena (e.g., agreement, scrambling, nominalization) [13]. Since our models are sentence embedding encoders (rather than generative language models), we evaluate a centroid-based acceptability proxy: for each phenomenon file, we compute the centroid of the grammatical sentence embeddings, score each sentence by cosine similarity to this centroid, and count a minimal pair as correct if the grammatical sentence receives a higher score than its ungrammatical counterpart. We report the resulting pairwise accuracy per phenomenon (in %) in Table 7. Following the visualization convention used in the TurBLiMP paper, cell background colors indicate relative performance, ranging from lower (red) to higher (green).

Table 7: TurBLiMP centroid-based pairwise accuracy by linguistic phenomenon.

	TurkishTokenizer	Mursit	CosmosGPT2	Tabi
Anaphor Agreement	**52.3**	48.6	49.6	49.6
Argument Str. Tran.	47.4	54.7	**57.0**	56.3
Argument Str. Ditr.	51.0	36.4	**59.3**	51.4
Binding	**86.7**	70.7	70.5	63.6
Determiners	**99.7**	96.3	93.6	58.4
Ellipsis	**62.9**	49.3	47.3	55.4
Irregular Forms	**54.9**	32.7	33.3	45.9
Island Effects	**93.1**	49.9	35.7	50.7
Nominalization	45.4	51.2	51.7	**52.1**
NPI Licensing	**71.9**	46.8	48.7	48.3
Passives	41.9	54.4	43.1	**61.5**
Quantifiers	25.5	**44.9**	21.1	19.8
Relative Clauses	48.4	48.7	**51.5**	49.4
Scrambling	**59.1**	56.5	55.3	55.6
Subject Agreement	**68.4**	56.9	55.1	60.8
Suspended Affixation	**71.2**	45.4	40.4	47.3
Model Average	**61.2**	52.7	50.8	51.6

Across many categories, TurkishTokenizer improves the separation of grammatical vs. ungrammatical minimal pairs under this proxy. A complementary evaluation of true grammatical sensitivity would require scoring minimal pairs with language model likelihood or a supervised acceptability classifier, which we leave for future work.

5 Future Work

This study highlights the importance of linguistic integrity and computational efficiency in tokenization, presenting a framework to guide the development of tokenizers optimized for morphologically rich and low-resource languages. Despite these promising results, much work remains to unlock the full potential of tokenizers. Future improvements will focus on incorporating advanced morphological analysis steps, which will further enhance their capability to capture the rich grammatical and semantic structures of Turkish. These steps may include integrating more sophisticated linguistic rules, handling rare morphemes, and accounting for contextual variations that impact tokenization in complex languages. Such enhancements will not only improve linguistic fidelity but also expand the scope of the tokenizers for diverse NLP applications.

Future work will also explore adapting the tokenizer to additional languages. Extending the approach beyond Turkish requires constructing language-specific lexical resources (e.g., root and affix inventories) and corresponding decoding and normalization rules, and validating the resulting tokenizers on language-appropriate benchmarks.

Although still in the early stages of development, this tokenizer provides a strong foundation for further innovation. Its initial performance gives hope that, with targeted improvements, it can evolve into a robust, versatile tool for tokenizing morphologically rich languages. By implementing these additional steps and conducting further evaluations across languages and tasks, this research aims to establish a new standard for linguistically informed tokenization, ultimately advancing the quality and efficiency of language models in a wide array of applications.

6 Conclusion

We presented a linguistically informed, morphology-first hybrid tokenizer designed for Turkish and similar agglutinative languages. The tokenizer combines curated root and affix lexicons with

phonological normalization (mapping surface allomorphs to shared identifiers) and a controlled subword fallback for coverage. This design aims to produce token sequences that more closely align with morpheme boundaries while remaining practical for large-scale NLP pipelines.

On TR-MMLU, the proposed tokenizer achieves 90.29% TR % and 85.80% Pure %, indicating substantially stronger morpheme-level alignment than several general-purpose tokenizers. We additionally report downstream sentence embedding evaluation on STS and TR-MTEB using **randomly initialized** models to isolate tokenizer effects from pretrained knowledge. The TurkishTokenizer-based model reaches **51.44%** Pearson correlation on STSb-TR, compared to 43.01% for the Tabi baseline-a gain of **+8.43 percentage points**. On TR-MTEB, TurkishTokenizer achieves 39.57% overall average compared to 34.92% for Tabi (+4.65 points). These gaps demonstrate that morphology-first tokenization provides a stronger inductive bias for learning Turkish semantic representations from scratch, and the same tokenizer also yields the strongest average accuracy on a centroid-based TurBLiMP minimal-pairs proxy.

We emphasize that empirical claims in this paper are Turkish-focused. We outline concrete next steps-improved morphophonological handling, better capitalization edge cases, and standardized efficiency measurements-in Section 5.

References

[1] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach, July 2019. arXiv:1907.11692 [cs].

[2] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1715–1725, Berlin, Germany, August 2016. Association for Computational Linguistics.

[3] Mike Schuster and Kaisuke Nakajima. Japanese and korean voice search. In *2012 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 5149–5152, March 2012.

[4] Taku Kudo and John Richardson. SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 66–71, Brussels, Belgium, November 2018. Association for Computational Linguistics.

[5] Cagri Toraman, Eyup Halit Yilmaz, Furkan Şahinuç, and Oguzhan Ozcelik. Impact of tokenization on language models: An analysis for turkish. *ACM Transactions on Asian and Low-Resource Language Information Processing*, 22(4):1–21, April 2023. arXiv:2204.08832 [cs].

[6] Yiğit Bekir Kaya and A. Cüneyd Tantuğ. Effect of tokenization granularity for turkish large language models. *Intelligent Systems with Applications*, 21:200335, March 2024.

[7] M. Ali Bayram, Ali Arda Fincan, Ahmet Semih Gümüş, Sercan Karakaş, Banu Diri, and Savaş Yıldırım. Tokenization standards for linguistic integrity: Turkish as a benchmark, February 2025. arXiv:2502.07057 [cs].

[8] M. Ali Bayram, Ali Arda Fincan, Ahmet Semih Gümüş, Banu Diri, Savaş Yıldırım, and Öner Aytaş. Setting standards in turkish nlp: Tr-mmlu for large language model evaluation, January 2025. arXiv:2501.00593 [cs].

[9] H Toprak Kesgin, M Kaan Yuce, Eren Dogan, M Egemen Uzun, Atahan Uz, H Emre Seyrek, Ahmed Zeer, and M Fatih Amasyali. Introducing cosmosgpt: Monolingual training for turkish language models. *arXiv preprint arXiv:2404.17336*, 2024.

[10] Özgür Uğur, Mahmut Göksu, Mahmut Çimen, Musa Yılmaz, Esra Şavirdi, Alp Talha Demir, Rumeysa Güllüce, İclal Çetin, and Ömer Can Sağbaş. Mecellem models: Turkish models trained from scratch and continually pre-trained for the legal domain. *arXiv preprint arXiv:2601.16018*, January 2026.

[11] Melikşah Türker, A. Ebrar Kızıloğlu, Onur Güngör, and Susan Üsküdarlı. TabiBERT: A Large-Scale ModernBERT Foundation Model and A Unified Benchmark for Turkish, January 2026. arXiv:2512.23065 [cs].

[12] Mehmet Selman Baysan and Tunga Gungor. TR-MTEB: A comprehensive benchmark and embedding model suite for Turkish sentence representations. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 8867–8887, Suzhou, China, November 2025. Association for Computational Linguistics.

[13] Ezgi Başar, Francesca Padovani, Jaap Jumelet, and Arianna Bisazza. TurBLiMP: A Turkish Benchmark of Linguistic Minimal Pairs. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 16495–16510, Suzhou, China, November 2025. Association for Computational Linguistics.

[14] Taku Kudo. Subword regularization: Improving neural network translation models with multiple subword candidates, April 2018. arXiv:1804.10959 [cs].

[15] Batuhan Baykara and Tunga Güngör. Abstractive text summarization and new large-scale datasets for agglutinative languages turkish and hungarian. *Language Resources and Evaluation*, 56(3):973–1007, September 2022.

[16] Yirong Pan, Xiao Li, Yating Yang, and Rui Dong. Morphological word segmentation on agglutinative languages for neural machine translation, January 2020. arXiv:2001.01589 [cs].

[17] Matthias Huck, Simon Riess, and Alexander Fraser. Target-side word segmentation strategies for neural machine translation. In *Proceedings of the Second Conference on Machine Translation*, pages 56–67, September 2017.

[18] Nihal Zuhal Kayalı and Sevinç İlhan Omurca. Hybrid tokenization strategy for turkish abstractive text summarization. In *2024 8th International Artificial Intelligence and Data Processing Symposium (IDAP)*, pages 1–6, September 2024.

[19] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics.

[20] Peter Henderson, Jieru Hu, Joshua Romoff, Emma Brunskill, Dan Jurafsky, and Joelle Pineau. Towards the systematic reporting of the energy and carbon footprints of machine learning, November 2022. arXiv:2002.05651 [cs].

[21] Pedro Henrique Martins, Patrick Fernandes, João Alves, Nuno M. Guerreiro, Ricardo Rei, Duarte M. Alves, José Pombal, Amin Farajian, Manuel Faysse, Mateusz Klimaszewski, Pierre Colombo, Barry Haddow, José G. C. de Souza, Alexandra Birch, and André F. T. Martins. Eurollm: Multilingual language models for europe, September 2024. arXiv:2409.16235 [cs].

[22] Zhenghao Lin, Zhibin Gou, Yeyun Gong, Xiao Liu, Yelong Shen, Ruochen Xu, Chen Lin, Yujiu Yang, Jian Jiao, Nan Duan, and Weizhu Chen. Not all tokens are what you need for pretraining. In *Proceedings of the 38th International Conference on Neural Information Processing Systems*, NIPS '24, Red Hook, NY, USA, 2024. Curran Associates Inc.

[23] Martin Neubeck. Flexible byte-pair tokenizers for dynamic vocabulary. Technical Report, 2024.

[24] Valentin Hofmann, Janet Pierrehumbert, and Hinrich Schütze. Superbizarre is not superb: Derivational morphology improves bert's interpretation of complex words. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics*, pages 3594–3608, 2021.

[25] Haris Jabbar. Morphpiece: A linguistic tokenizer for large language models, February 2024. arXiv:2307.07262 [cs].

[26] Elshad Rahimov. miLLi: Model Integrating Local Linguistic Insights for Morphologically Robust Tokenization, December 2025.

[27] Ehsaneddin Asgari, Yassine El Kheir, and Mohammad Ali Sadraei Javaheri. MorphBPE: A Morpho-Aware Tokenizer Bridging Linguistic Complexity for Efficient LLM Training Across Morphologies, February 2025. arXiv:2502.00894 [cs].

[28] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.

[29] Mete Ismayilzada, Defne Circi, Jonne Sälevä, Hale Sirin, Abdullatif Köksal, Bhuwan Dhingra, Antoine Bosselut, Duygu Ataman, and Lonneke van der Plas. Evaluating morphological compositional generalization in large language models, February 2025. arXiv:2410.12656 [cs].

[30] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *Proceedings of the 9th International Conference on Learning Representations (ICLR)*, 2021.

[31] Mohamed Rashad. Arabic-nougat: Fine-tuning vision transformers for arabic ocr and markdown extraction, November 2024. arXiv:2411.17835 [cs].

[32] Rudolf Rosa and Ivana Kvapilíková. A benchmark for scandinavian tokenizers. Technical Report, 2024.

[33] Henrique Schechter Vera, Sahil Dua, Biao Zhang, Daniel Salz, Ryan Mullins, Sindhu Raghuram Panyam, Sara Smoot, Iftekhar Naim, Joe Zou, Feiyang Chen, Daniel Cer, Alice Lisak, Min Choi, Lucas Gonzalez, Omar Sanseviero, Glenn Cameron, Ian Ballantyne, Kat Black, Kaifeng Chen, Weiyi Wang, Zhe Li, Gus Martins, et al. Embeddinggemma: Powerful and lightweight text representations, 2025.

[34] Figen Beken Fikri, Kemal Oflazer, and Berrin Yanikoglu. Semantic Similarity Based Evaluation for Abstractive News Summarization. In *Proceedings of the First Workshop on Natural Language Generation, Evaluation, and Metrics (GEM)*, pages 24–33, Online, August 2021. Association for Computational Linguistics.